

Configuration Manual

Trustworthy Deep Learning for Pharmaceutical Tablet Quality Prediction:
End-to-End Multi-Task Modelling with Dual Uncertainty Quantification on
Industrial Compression Trajectories

MSc (Research) Practicum
Artificial Intelligence

Arpit Joshua Elias
Student ID: 24257567

School of Computing
National College of Ireland

Supervisor: Arundev Vamadevan

August 2026

1 Purpose and scope

This manual covers all the things which someone would need to rerun all the experiments behind the research report. That means the software and its versions along with the hardware and how the dataset is obtained and then assembled, how the series of the notebooks they go in, and model and evaluation settings, and then how to start the demonstration application as well.

Installation of a standard software is not covered, as the guidance for this deliverable asks. Git, Python and a browser is expected and assumed to be working already, along with a person having a kaggle account that has the access to the T4 tesla GPU access. What follows is only the part specific to this project.

All code, result files and notebooks referenced here are available in the public repository at <https://github.com/arpitelias/mt-trajnet-thesis>. The demonstration application is deployed at <https://mt-trajnet-thesis-4kznhd8sjhv28lcfds4e2k.streamlit.app/>.

1.1 Who this research benefits

Three of the groups they stand to gain here. The quality staff and the process engineers on a tablet line, they get a signal while the batch is still fresh rather than after the lab has finished, plus a flag as well when the model is looking at a product it does not actually recognise. Quality by the design programmes get a worked example of tying a critical quality attribute back to the process signals that actually drive it, using the trace the press already records. The researchers working on the trustworthy ML for industrial data get an evaluation that is honest and true where the model holds up and where it does not, on a public dataset anyone can actually rerun.

None of this replaces the lab testing, It sits alongside the existing quality process as a triage signal.

2 Hardware and environment

Two of the environments were used. The preprocessing audits, the figures and the exploratory analysis were all done on a local windows machine, and everything that needed a use of GPU for calibration, training, attribution and ablations, they were done on Kaggle, which gives a free Tesla T4 and a container image that does not drift between the sessions.

Table 1: Execution environments.

	Local machine	Kaggle notebook
Role	EDA, data audit, figure generation, report and application development	Model training, calibration, attribution, ablations, weight export
Operating system	Windows 11	Linux (Kaggle container)
Processor	<i>see note below</i>	Intel Xeon (2 vCPU allocated)
Memory	<i>see note below</i>	13 GB RAM
Accelerator	None (CPU only)	NVIDIA Tesla T4, 16 GB
Session limit	Not applicable	12 hours, 30 GB disk

Nothing in this needs more than a single GPU as the longest single training that ran, finished under 10 minutes, and so the free Kaggle tier was enough for the whole project.

A GPU is only needed to just rerun the training notebooks. Every result file is committed to the repository on which the application depends on, so the analysis can be read without the use of a GPU.

2.1 Software versions

Table 2 lists what the project depends on. Kaggle updates its container image from time to time, so rather than trusting a version written down months ago, record the versions from the session you actually use. Listing 1 does that in one cell.

Table 2: Software dependencies. Versions are those recorded in the Kaggle environment used for the final experimental runs.

Package	Version	Used for
Python	3.11	Interpreter for all notebooks and the application
PyTorch	2.x	MT-TrajNet, evidential heads, all training and inference
NumPy	1.26	Array handling, fold construction, metric computation
pandas	2.x	Sensor file loading, feature tables, result assembly
scikit-learn	1.x	LASSO baseline, grouped splits, RMSE and R^2
XGBoost	2.x	Gradient boosting baseline, default and tuned
SciPy	1.x	Kendall’s tau, bootstrap intervals, paired tests
SHAP	0.4x	Gradient explainer for the attribution analysis
Matplotlib	3.x	All figures in the report
seaborn	0.13	Exploratory distribution plots
Streamlit	≥ 1.32	Demonstration application
Plotly	≥ 5.18	Interactive charts in the application

Listing 1: Recording the exact environment versions.

```
import sys, torch, numpy, pandas, sklearn, scipy, xgboost, shap, matplotlib
print("python ", sys.version.split()[0])
print("torch ", torch.__version__, "| CUDA:", torch.cuda.is_available())
print("GPU ", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "none")
for m in (numpy, pandas, sklearn, scipy, xgboost, shap, matplotlib):
    print(f"{m.__name__:12s}", m.__version__)
```

3 Data acquisition and preparation

3.1 Source dataset

The data comes from Žagar and Mihelič (2022), published in *Scientific Data* under a CC BY 4.0 licence and archived on figshare at <https://doi.org/10.1038/s41597-022-01203-x>. It is too large to sit in the repository, so download it from there.

After download the following files are required:

- Twenty-five per-product sensor files, named `1.csv` to `25.csv`, holding the ten-channel time series for every batch of that product code.
- `Laboratory.csv`, holding the four laboratory quality measurements per batch.
- `Process.csv` and `Normalization.csv`, used during the initial audit only.

3.2 Assembly and integrity verification

The twenty-five sensor files are joined to the laboratory measurements on the batch identifier, then serialised into one file, `assembled_trajectories.pkl`, holding 1,005 trajectories

with their targets and product-code labels. I audited that join and it matched 1,005 to 1,005, with nothing orphaned on either side.

Before doing anything else, every notebook checks the MD5 fingerprint of that file. A changed, truncated or simply wrong input then fails on the spot, instead of turning up two notebooks later as a number nobody can account for. The expected value is:

Listing 2: Data integrity assertion, run at the start of every notebook.

```
import hashlib
h = hashlib.md5()
with open(DATA + "/assembled_trajectories.pkl", "rb") as fh:
    for chunk in iter(lambda: fh.read(1 << 20), b''):
        h.update(chunk)
assert h.hexdigest() == "7d7fc76be1e4940198b76d9d0797a3a9", \
    "stop: this is not the authoritative dataset"
```

3.3 Uploading to Kaggle

The assembled file is uploaded once as a private Kaggle dataset and attached to each notebook. The path used throughout is:

```
/kaggle/input/datasets/arpitjoshua/mt-trajnet-thesis-data/kaggle_upload/assembled_trajectories.pkl
```

Re-upload it under another account or name and this path is the one line each notebook needs changed. The trained fold weights (`mttrajnet_fold1.pt`, and the same for folds 2 and 3) live in that same Kaggle dataset. They are kept out of Git because of their size, which is what the `*.pt` rule in `.gitignore` is for.

4 Repository structure

Table 3: Repository layout.

Path	Contents
<code>notebooks/</code>	Twenty notebooks covering audit, EDA, preprocessing, baselines, model development, calibration, attribution, out-of-distribution testing, ablations and figures
<code>results/</code>	Every result file cited in the report, as JSON, CSV and NPZ, plus the nine figures in <code>results/figures/</code>
<code>mt-trajnet-app/</code>	The Streamlit demonstration application and its precomputed data
<code>proposal/</code>	The CA2 proposal and supporting design documents
<code>guidelines/</code>	College templates, marking rubric and submission guidance
<code>deliverables/</code>	Weekly submissions made to the supervisor

5 Execution order

The notebooks, they are numbered in the order they were written, but there is no need to run all of them. Sequence that regenerates every number in the report is given in Table 4. Development trail is in the notebooks 1-7, including the things which were then later on corrected as well. They are just there for transparency and not because the final results lean on them.

What matters from the rest is that one point: **notebook 08 supersedes notebooks 02, 03 and 06**. Plain grouped slit were used by the earlier ones, which then dropped 90 of the 98 high hardness batches into one of the test fold and which then overall inflated the hardness error. The folds with the cluster balanced assignment is rebuilt in the notebook

8. Each and every result in the study comes from that or from a notebook that loads its outputs.

Table 4: Execution order for reproducing the reported results. Runtimes are for a single Kaggle Tesla T4 session.

#	Notebook	Produces	Runtime
1	Audit	Join verification, 1,005 to 1,005, no orphans	< 2 min, CPU
2	EDA	Trajectory lengths, target distributions, the bimodal hardness finding	< 5 min, CPU
3	01_preprocessing_and_baselines	Engineered features, LASSO and XGBoost baselines, <code>comparison_stratified.csv</code>	$\approx$ 10 min, CPU
4	08-final-stratified	MT-TrajNet on corrected folds, <code>mttrajnet_stratified.json</code> , <code>oof_predictions.npz</code> , fold weights	$\approx$ 25 min, GPU
5	09-stratified-analysis	<code>rq1_stratified.json</code> , corrected paired tests against baselines	< 5 min, CPU
6	10-rq2-stratified	<code>rq2_stratified.json</code> , single-task comparison and parameter counts	$\approx$ 25 min, GPU
7	11-shap-all-targets	<code>rq3b_all_targets_stratified.json</code> , uncertainty-stratified attribution	$\approx$ 15 min, GPU
8	12-ood-test	<code>ood_test.json</code> , code 15 against code 25	< 5 min, GPU
9	13-extended-metrics	<code>extended_metrics.json</code> , pooled metrics, coverage, leave-one-code-out	< 5 min, CPU
10	15-rq3-deep-ensemble	<code>rq3_ensemble_comparison.json</code> , five-seed ensemble reference	$\approx$ 13 min, GPU
11	16-point-output-ablation	<code>point_output_ablation.json</code> , evidential versus plain output	$\approx$ 6 min, GPU
12	17-export-fold1-weights	<code>app_data.json</code> and <code>trajectories.json</code> for the application	$\approx$ 5 min, GPU
13	14-figures	All nine figures in <code>results/figures/</code>	< 5 min, CPU

A full rerun costs about ninety minutes of GPU time, well inside a single Kaggle session.

6 Model and training configuration

6.1 Preprocessing settings

Table 5: Preprocessing configuration.

Setting	Value	Note
Downsampling stride	2	Halves the length, keeps the signal shape
Sequence length	6,000 steps	Zero-padded if shorter, truncated if longer
Input channels	10	All sensor channels kept
Channel scaling	zero mean, unit variance	Fitted on training folds only
Target scaling	zero mean, unit variance	Fitted per fold, inverted for reporting

The order of channel is fixed and has to stay that way as the attribution results are reported per channel. Getting this wrong relabels the findings rather than just breaking anything, Which then makes it a kind of error that is really easy to miss. The

order is: `tbl_speed`, `fom`, `main_comp`, `tbl_fill`, `SREL`, `pre_comp`, `cyl_main`, `cyl_pre`, `stiffness`, `ejection`.

6.2 Architecture

MT-TrajNet is a shared dilated temporal convolutional encoder feeding four evidential regression heads. Table 6 gives the configuration; the total parameter count is 384,400.

Table 6: MT-TrajNet architecture configuration.

Component	Configuration
Encoder blocks	4 residual blocks, dilation 1, 2, 4, 8
Per block	2 causal convolutions, kernel width 3
Channels	128 throughout
Activation, dropout	ReLU, dropout 0.1
Residual path	1×1 convolution on the first block to match width
Pooling	Global average over time, giving 128 dimensions
Heads	4 two-layer perceptrons, hidden width 64
Head output	4 parameters per target: γ , ν , α , β
Clamping	ν , α , β clamped to $[10^{-2}, 10^3]$
Total parameters	384,400

The clamping is worth attention. Leave it out and the variance term can collapse, after which the uncertainty values come out meaningless. Anyone reimplementing this needs the clamp inside the head’s forward pass, applied after the softplus and after adding one to α .

6.3 Training settings

Hyperparameters were fixed, not searched. The report lists a nested search as future work. To reproduce the numbers as published, use exactly the values below.

Table 7: Training configuration. These values are fixed, not searched.

Setting	Value	Note
Optimiser	Adam	
Learning rate	5×10^{-4}	
Batch size	16	Set by the 6,000-step sequences
Maximum epochs	150	
Early stopping	patience 10	On validation evidential loss
Gradient clipping	norm 1.0	
Loss	sum of 4 evidential losses	Equal weights, targets standardised
Evidence regulariser	1.0	
Random seed	42	
Determinism	<code> cudnn.deterministic =True</code>	With <code>benchmark=False</code>

7 Evaluation configuration

7.1 Fold assignment

Cross-validation is grouped by product code so no code appears on both sides of a split. The fold assignment is fixed and must be reproduced exactly:

Listing 3: Fold assignment. Code 15 is excluded from cross-validation entirely.

```
fold_codes = [[1, 2, 4, 6, 7, 9, 13, 18, 25],
               [10, 11, 17, 19, 22, 24],
               [3, 5, 8, 12, 14, 16, 20, 21, 23]]
# code 15 (64 batches) withheld as the out-of-distribution probe
# code 25 (34 batches) acts as the matched in-regime control
# 941 batches remain inside the three-fold cross-validation
```

That assignment spreads the high-hardness cluster to roughly 22, 6 and 6 batches per test fold. A plain GroupKFold gives 90, 6, 2 instead, and will not reproduce the hardness figures in the report.

7.2 Calibration

Uncertainty is calibrated by cross-fitting. A fold's scale factor comes only from the out-of-fold predictions of the other two folds, then gets applied to that fold, so no batch is ever calibrated against its own prediction. The scale multiplies:

Listing 4: Calibrated interval construction.

```
scale_per_fold = {"dissolution_av": 0.228, "tbl_av_hardness": 0.327,
                  "tbl_rsd_weight": 0.572, "fct_tensile": 0.214} # fold 1
std_calibrated = std_raw * scale_per_fold[target]
lower = prediction - 1.645 * std_calibrated # 90% interval
upper = prediction + 1.645 * std_calibrated
```

Multiply, do not divide. Dividing blows the intervals up and gives coverage of 1.0 on every target, which is the tell that this has been done backwards.

7.3 Statistical testing and attribution

Model comparisons use the corrected resampled paired *t*-test of Nadeau and Bengio (2003), which allows for the dependence that overlapping training folds create.

Three folds leave that test with very little power. Anything non-significant is therefore reported as inconclusive, not as equivalence.

SHAP gradient explainer with 32 background samples over the full 314 batch fold 1 test set was done in the attribution analysis. Per target the batches were made to split into the least and most confident fifths by the model's own epistemic uncertainty. Per channel attributions were then averaged inside each of the group and then the two rankings compared with Kendall's tau.

8 Result files

9 User dashboard

The demo application is a Streamlit based dashboard covering the study in eight of the sections where there is an overview, the model, the results, calibration and uncertainty, interpretability, a batch explorer, and the limitations section as well. Navigation runs down the left sidebar, and each of the section open with the headline numbers for that part of the study.

9.1 Working with a single batch

The batch explorer is the part a process engineer would actually use as it takes one held out batch at a time and then puts the sensor trace, the four predictions with their calibrated intervals, and the measure lab value side by side. A selector moves between three of the groups of batches and the

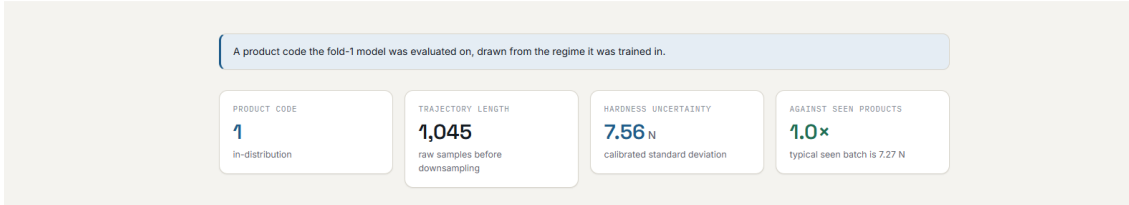
The batch explorer is the part a process engineer would use. It takes one held-out batch

Table 8: Contents of `results/`. Every number in the report traces to one of these files.

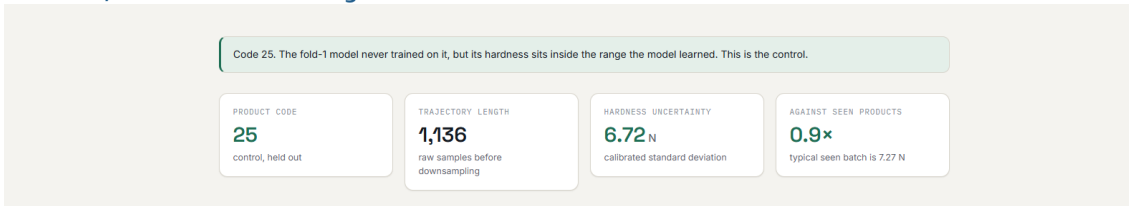
File	Contents
<code>mttrajnet_stratified.json</code>	Fold-mean and per-fold RMSE, coverage, ECE, uncertainty-error correlation, calibration scales
<code>comparison_stratified.csv</code>	RMSE for every baseline and MT-TrajNet, all four targets
<code>rq1_stratified.json</code>	Corrected paired test against tuned gradient boosting
<code>rq2_stratified.json</code>	Multi-task against single-task, with parameter counts
<code>rq3b_all_targets_stratified.json</code>	Kendall’s tau, p -values, per-channel attribution ratios
<code>ood_test.json</code>	Code 15 and code 25 against the cross-validation reference
<code>rq3_ensemble_comparison.json</code>	Five-seed deep ensemble against the evidential model
<code>point_output_ablation.json</code>	Point-output variant, isolating the evidential objective
<code>extended_metrics.json</code>	Pooled metrics with bootstrap intervals, coverage at three levels, leave-one-code-out
<code>oof_predictions.npz</code>	Per-batch out-of-fold predictions, uncertainties, fold RMSE
<code>fold_assignment.json</code>	The fold assignment and high-cluster distribution
<code>hardness_cluster_finding.json</code>	The bimodal hardness analysis
<code>shap_norm.npy</code>	Normalisation statistics used by the attribution notebook

at a time and puts the sensor trace, the four predictions with their calibrated intervals, and the measured Figure 1 shows what that does to the hardness uncertainty.

Seen product · hardness uncertainty 7.56 N



Code 25, unseen but same regime · 6.72 N



Code 15, unseen and shifted regime · 25.00 N

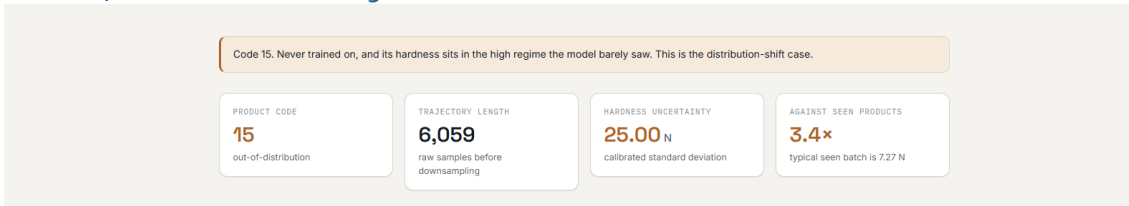


Figure 1: The batch explorer across three of the groups, also nothing tells the dashboard which code is unseen. The widening interval it comes from the model’s own uncertainty.

Reading these three together is the point of this dashboard. An unfamiliar product on its own cost nothings while and unfamiliar operating regime is flagged clearly and before anyone actos on the prediction.

9.2 Design decision: precomputed results

No model runs behind it. Every value on screen was computed once from the fold 1 model and written to two data files. That was deliberate:

- It removes PyTorch and the 1.5 MB weight file from the deployment, so the application starts instantly and runs on the free Streamlit tier without a GPU.
- Every displayed number traces to a verified result file rather than to a fresh model run that might differ.
- There is no model to fail during a live demonstration.

Before any of those files were written, the saved weights were checked against the fold 1 RMSE (3.344, 7.900, 0.770, 0.404) and the per-code uncertainty. The application inputs were only generated once the weights reproduced those numbers exactly.

9.3 Application structure and data

Table 9: Application layout.

Path	Contents
app.py	Entry point and section routing
config/theme.py	Palette, typography and shared chart styling
components/	Reusable cards, callouts and chart builders
utils/data_loader.py	Cached data access, target and channel naming
pages_/	One module per section
data/results_bundle.json	Study-level results, assembled from <code>results/</code> by <code>build_bundle.py</code>
data/app_data.json	Predictions, uncertainties and intervals for 378 held-out batches
data/trajectories.json	Downsampled sensor curves for 30 representative batches
.streamlit/config.toml	Pinned light theme, so the interface renders identically on any machine

9.4 Running locally

Listing 5: Running the application on a local machine.

```
git clone https://github.com/arpitelias/mt-trajnet-thesis.git
cd mt-trajnet-thesis/mt-trajnet-app
pip install -r requirements.txt
streamlit run app.py
```

It opens at `http://localhost:8501`. On Windows the `streamlit` command is often not on the system path, in which case `python -m streamlit run app.py` works instead. The dependencies stop at Streamlit, Plotly, pandas and NumPy. PyTorch is not needed.

9.5 Deployment

Deployment runs on Streamlit Community Cloud straight from the repository, with the main file path set to `mt-trajnet-app/app.py`. Free-tier apps go to sleep after a while and take a few seconds to wake, so open the URL a minute or two before demonstrating it live.

10 Reproducibility notes

10.1 What is guaranteed

Given the same seed, the same data file and deterministic cuDNN flags, training on a Tesla T4 reproduces the reported numbers.

That was checked, not assumed. Notebook 16 was rerun on a different day and came back with identical fold RMSE values to three decimal places.

10.2 Known issues and how to resolve them

Table 10: Common issues encountered during this project and their resolution.

Symptom	Cause and resolution
<code>FileNotFoundError</code> on the pkl	The Kaggle input path differs by account. Locate it with a directory walk over <code>/kaggle/input</code> and update the <code>DATA</code> constant
<code>UnpicklingError</code> loading weights	PyTorch 2.6 defaults <code>weights_only</code> to <code>True</code> . Load saved bundles with <code>torch.load(path, weights_only=False)</code>
Uncertainty values far too large or small	The clamp on ν , α , β is missing from the head, or the softplus is applied twice. Compare a known code against its expected value before proceeding
Interval coverage of 1.0 on all targets	The calibration scale is being divided rather than multiplied
Hardness error much larger than reported	A plain <code>GroupKFold</code> has been used instead of the balanced assignment in Section 7
Kaggle session ends and variables are lost	Session memory clears when a notebook closes. Re-run the setup cells before continuing

10.3 Version control

Trained weights (`*.pt`) are kept out of the repository because of their size and live in the Kaggle dataset instead. Notebooks are committed with outputs intact, so the reported values can be read without rerunning anything, and every result file the report cites sits in `results/`.